UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN



Asignatura: Ingeniería de software II

Tema: Test Driven Development

Docente: Edgard Sarmiento Calizaya

Estudiantes:

- Condori Pallardelli, Emilio Alejandro

Arequipa - Perú

2026

1. Código base y criterio de interpretación

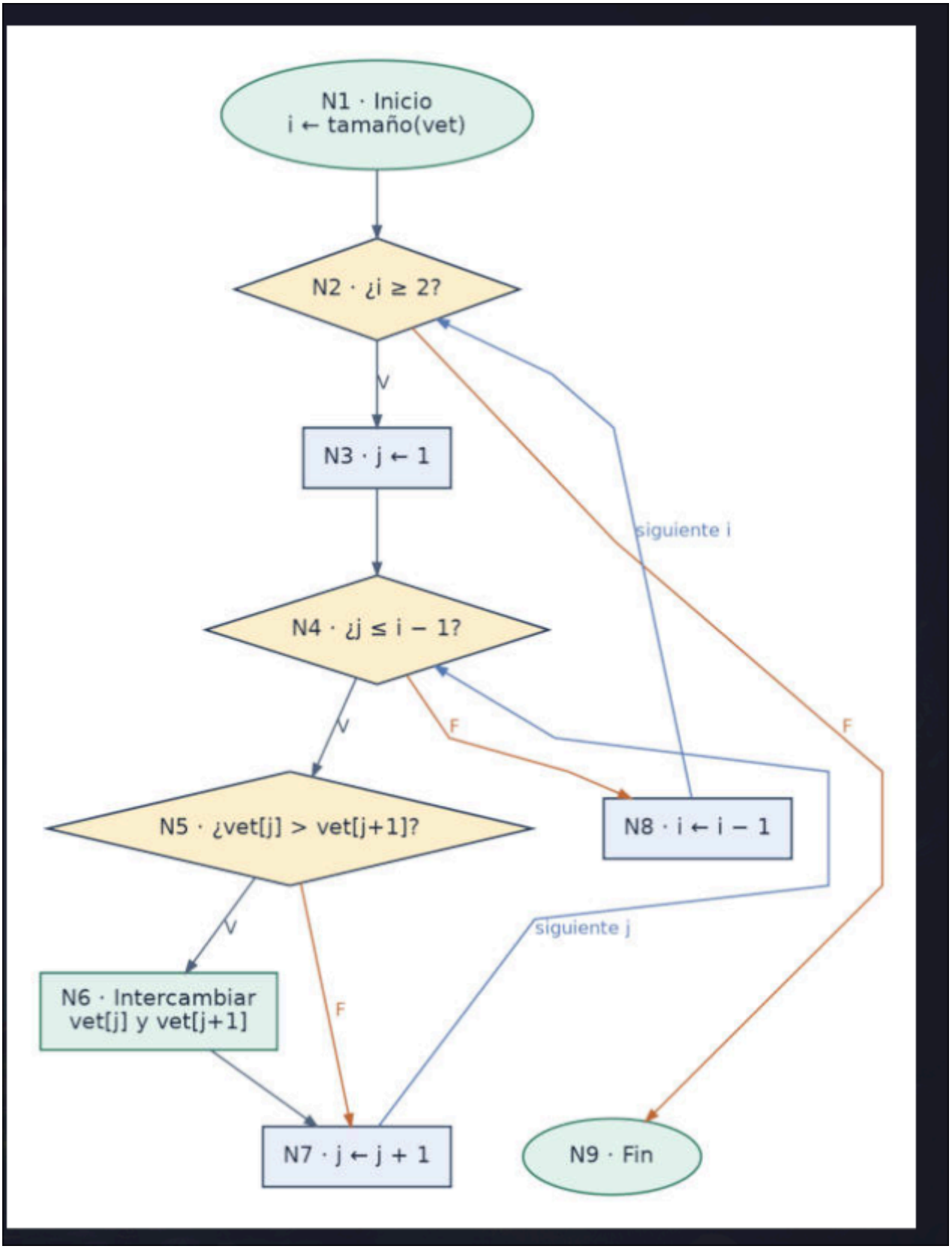
El código tomado en cuenta para las pruebas del bubble sort es el siguiente

```
bubbleSort(int vet[]) {
    integer i, j, aux;
    for (i=tamanho(vet); i>=2; i--) {
        for (j=1; j<=i-1; j++) {
            if (vet[j] > vet[j+1]) {
                aux=vet[j];
                vet[j]=vet[j+1];
                vet[j+1]=aux;
            }
        }
    }
}
```

El pseudocodigo usa indices desde 0, sin embargo en python se emplea desde 0, por lo que la implementacion debe de mantener el comportamiento esperado y las evaluaciones indicadas

2. Actividad 1

El grafo distingue las tres decisiones (rombos) y los dos retornos de los bucles. V representa una evaluación verdadera y F una falsa. Se identifican nueve nodos y once aristas visibles.


N1 · Inicio
i ← tamaño(vet)
N2 · ¿i ≥ 2?
V
N3 · j ← 1
siguiente i
N4 · ¿j ≤ i − 1?
V
F
F
N5 · ¿vet[j] > vet[j+1]?
N8 · i ← i − 1
V
siguiente j
N6 · Intercambiar
vet[j] y vet[j+1]
F
N7 · j ← j + 1
N9 · Fin

NODO	OPERACIÓN	NODO	OPERACIÓN
N1	Inicio; i ← tamaño(vet)	N2	¿i ≥ 2?
N3	j ← 1	N4	¿j ≤ i − 1?
N5	¿vet[j] > vet[j+1]?	N6	Intercambiar
N7	j ← j + 1	N8	i ← i − 1
N9	Fin		

Aristas: N1→N2; N2→N3/N9; N3→N4; N4→N5/N8; N5→N6/N7; N6→N7; N7→N4; N8→N2.

3. Actividad 2 | Complejidad ciclomática

Para un único componente conexo del grafo de flujo de control se emplea V(G) = E − N + 2P.

VARIABLE	VALOR	JUSTIFICACIÓN
E (aristas)	11	Conexiones dirigidas entre nodos
N (nodos)	9	N1, N2, …, N9
P (componentes)	1	Un único subgrafo conectado
V(G)	11 − 9 + 2(1) = 4	Resultado de complejidad ciclomática

Verificación por predicados: existen tres nodos de decisión, N2, N4 y N5; por tanto V(G) = 3 + 1 = 4 entonces su complejidad ciclomatica es 4.

4. Actividad 3

Se elige cobertura de decisiones + condición simple. Como cada decisión contiene un único predicado booleano, basta con obtener V y F en los nodos N2, N4 y N5. Los cuatro caminos se ejecutan con entradas factibles.

CAMINO	RASGO QUE LO DISTINGUE	ENTRADA
CP1	Salida directa N2→N9; no entra al for externo (lista de 1 solo elemento)	[7]
CP2	Incluye N5→N7 (condición de intercambio F) (una lista ya ordenada)	[1, 2]
CP3	Incluye N5→N6 (intercambio V) (un par de elementos desordenados requiere intercambio	[2, 1]
CP4	Repite el ciclo interno N7→N4 y externo N8→N2 (para una lista desordenada repetir el ordenamiento)	[2, 3, 1]

Secuencias observadas de nodos

CP1 N1 → N2(F) → N9

CP2 N1 → N2(V) → N3 → N4(V) → N5(F) → N7 → N4(F) → N8 → N2(F) → N9

CP3 N1 → N2(V) → N3 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(F) → N9

CP4 N1 → N2(V) → N3 → N4(V) → N5(F) → N7 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(V) → N3 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(F) → N9

Los paréntesis(V/F) representan la salida para cada evaluación en su respectivo caso de prueba

5. Actividad 4

Las entradas y salidas de la tabla fueron verificadas con una implementación ejecutable del algoritmo. Los caminos corresponden al grafo

```python
@dataclass
class Traza:
    """Registro de nodos visitados y resultados de las condiciones."""

    nodos: List[str] = field(default_factory=list)
    decisiones: List[Tuple[str, str]] = field(default_factory=list)

    def visitar(self, nodo: str) -> None:     ■ Missing function or method docstring (missing-function-docstring)
        self.nodos.append(nodo)

    def evaluar(self, nodo: str, condicion: bool) -> bool:     ■ Missing function or method docstring (missing-function-docstring)
        self.visitar(nodo)
        self.decisiones.append((nodo, "V" if condicion else "F"))
        return condicion


def bubble_sort(vet: List[int], traza: Optional[Traza] = None) -> None:     ■ Redefining name 'traza' from outer scope (line 60) (r
    """Ordena *vet* de menor a mayor in situ.

    No devuelve otra lista. La traza es opcional y no altera el ordenamiento.
    """
    t = traza if traza is not None else Traza()
    t.visitar("N1")
    i = len(vet)

    while t.evaluar("N2", i >= 2):
        t.visitar("N3")
        j = 1  # Índice lógico inicial del pseudocódigo (base 1).

        while t.evaluar("N4", j <= i - 1):
            if t.evaluar("N5", vet[j - 1] > vet[j]):
                t.visitar("N6")
                aux = vet[j - 1]
                vet[j - 1] = vet[j]
                vet[j] = aux
            t.visitar("N7")
            j += 1

        t.visitar("N8")
        i -= 1

    t.visitar("N9")
```

Tests escritos para verificar la implementación, segun los casos de prueba indicados anteriormente

```
def test_CP1_sin_entrar_al_bucle_externo(self):     ■■ Missing function or r
    self.verificar(
        [7], [7],
        ["N1", "N2", "N9"],
        [("N2", "F")],
    )

def test_CP2_comparacion_sin_intercambio(self):     ■■ Missing function or r
    self.verificar(
        [1, 2], [1, 2],
        ["N1", "N2", "N3", "N4", "N5", "N7", "N4", "N8", "N2", "N9"],
        [("N2", "V"), ("N4", "V"), ("N5", "F"), ("N4", "F"), ("N2", "F")],
    )

def test_CP3_comparacion_con_intercambio(self):     ■■ Missing function or r
    self.verificar(
        [2, 1], [1, 2],
        ["N1", "N2", "N3", "N4", "N5", "N6", "N7", "N4", "N8", "N2", "N9"],
        [("N2", "V"), ("N4", "V"), ("N5", "V"), ("N4", "F"), ("N2", "F")],
    )

def test_CP4_repeticion_de_bucles(self):     ■■ Missing function or method c
    self.verificar(
        [2, 3, 1], [1, 2, 3],
        ["N1", "N2", "N3", "N4", "N5", "N7", "N4", "N5", "N6", "N7",
         "N4", "N8", "N2", "N3", "N4", "N5", "N6", "N7", "N4", "N8",
         "N2", "N9"],
        [("N2", "V"), ("N4", "V"), ("N5", "F"), ("N4", "V"), ("N5", "V"),
         ("N4", "F"), ("N2", "V"), ("N4", "V"), ("N5", "V"), ("N4", "F"),
         ("N2", "F")],
    )
```

```
class TestRegresion(unittest.TestCase):     ■ Miss
    def test_lista_vacia(self):     ■ Missing func
        datos = []
        bubble_sort(datos)
        self.assertEqual(datos, [])

    def test_repetidos(self):     ■ Missing functi
        datos = [2, 2, 1, 1]
        bubble_sort(datos)
        self.assertEqual(datos, [1, 1, 2, 2])

    def test_valores_negativos_y_cero(self):     ■
        datos = [0, -3, 5, -1]
        bubble_sort(datos)
        self.assertEqual(datos, [-3, -1, 0, 5])

    def test_orden_inverso(self):     ■ Missing fu
        datos = [5, 4, 3, 2, 1]
        bubble_sort(datos)
        self.assertEqual(datos, [1, 2, 3, 4, 5])
```

Resultados obtenidos de los tests

```
ID CP,Camino en el grafo,Valores de prueba,Resultado esperado,Objetivo del cas
CP1,N1-N2(F)-N9,[7],[7],No entra al bucle externo
CP2,N1-N2(V)-N3-N4(V)-N5(F)-N7-N4(F)-N8-N2(F)-N9,"[1, 2]","[1, 2]",No realiza
CP3,N1-N2(V)-N3-N4(V)-N5(V)-N6-N7-N4(F)-N8-N2(F)-N9,"[2, 1]","[1, 2]",Realiza
CP4,N1-N2(V)-N3-N4(V)-N5(F)-N7-N4(V)-N5(V)-N6-N7-N4(F)-N8-N2(V)-N3-N4(V)-N5(V)
```

ID CP	CAMINO EN EL GRAFO	VALORES DE PRUEBA	RESULTADO ESPERADO
CP1	N1→N2(F)→N9	[7]	[7]
CP2	N1→N2(V)→N3→N4(V)→N5(F)→N7→N4(F)→N8→N2(F)→N9	[1, 2]	[1, 2]

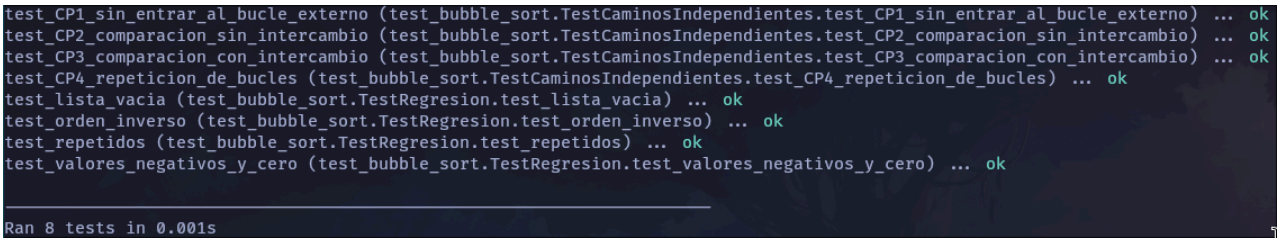
ID CP	CAMINO EN EL GRAFO	VALORES DE PRUEBA	RESULTADO ESPERADO
CP3	N1→N2(V)→N3→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(F)→N9	[2, 1]	[1, 2]
CP4	N1→N2(V)→N3→N4(V)→N5(F)→N7→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(V)→N3→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(F)→N9	[2, 3, 1]	[1, 2, 3]

6. Comprobación de cobertura y ejecución

DECISIÓN	SALIDAS OBSERVADAS	COBERTURA
N2: $i \geq 2$	V y F	2/2
N4: $j \leq i - 1$	V y F	2/2
N5: vet[j] > vet[j+1]	V y F	2/2
Total	6 de 6 salidas	100 %

Ejecución de ocho pruebas unitarias con unittest: CP1–CP4 y cuatro pruebas complementarias (lista vacía, duplicados, números negativos/cero y orden inverso). Resultado: 8 pruebas ejecutadas; 8 exitosas; 0 fallidas.







5. Repositorio en github

